

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Numerisches Rechnen mit Computern ist nicht unproblematisch.

Das gilt für das Rechnen mit ganzzahligen Werten und insbesondere für das Rechnen mit Gleitkommawerten.

Diese Probleme sind keineswegs C-spezifisch, sondern treten bei allen Sprachen auf, die die entsprechenden Datentypen kennen. Der Grund dafür ist die begrenzte (feste) Anzahl an Stellen (bzw. Bits) mit der die Zahlen im Speicher des Rechners abgelegt werden.

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Dezimalzahl

Die Darstellung einer nicht negativen ganzen Zahl (*unsigned integer*) z als **Dezimalzahl** ist bekannt.

- Abbildung der Ziffern $\{0, \dots, 9\}$ auf ganze Zahlen. ✓
- Darstellung als Ziffernfolge der Länge n zur Basis 10. ✓

$$\text{dec}(z) = x_{n-1} \dots x_0$$

$$z = \sum_{i=0}^{n-1} x_i \cdot 10^i$$

$$\text{für } x_i \in \{0, \dots, 9\} \quad \text{mit } i = 0, \dots, n-1$$

Binärzahl

Die auf Hardware-Ebene verarbeiteten Daten, z.B. im Speicher hinterlegte Werte, werden **binär**, d.h. mit den Bits $\{0, 1\}$, codiert. $(\dots)_2$ kennzeichnet eine Bitfolge.

Vorzeichenlose Zahlen werden als **Binärzahlen** (Bitfolgen fester Länge n) codiert.

Für eine Dezimalzahl z ist $bin_n(z)$ die Bitfolge $(x_{n-1} \dots x_0)_2$ der Länge n , die, interpretiert als Binärzahl, z codiert.

$$z = \sum_{i=0}^{n-1} x_i \cdot 2^i$$

Wobei die Bits $\{0, 1\}$ mit den entsprechenden Dezimalzahlen identifiziert werden.

Beispiel

Beispiele

$$\textit{bin}_8(15) = ?$$

Beispiel

Beispiele

$$\text{bin}_8(15) = ?$$

$$\begin{aligned} 15 &= 8 + 4 + 2 + 1 \\ &= 2^3 + 2^2 + 2^1 + 2^0 \\ &= 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= (00001111)_2 \end{aligned}$$

$$\text{bin}_8(15) = (00001111)_2$$

Beispiel

Beispiele

$$\text{bin}_8(15) = ?$$

$$15 = 8 + 4 + 2 + 1$$

$$= 2^3 + 2^2 + 2^1 + 2^0$$

$$= 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

$$= (00001111)_2$$

$$\text{bin}_8(15) = (00001111)_2$$

$$\text{bin}_{32}(538623) = ?$$

Beispiel

Beispiele

$$\text{bin}_8(15) = ?$$

$$\begin{aligned} 15 &= 8 + 4 + 2 + 1 \\ &= 2^3 + 2^2 + 2^1 + 2^0 \\ &= 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= (00001111)_2 \end{aligned}$$

$$\text{bin}_8(15) = (00001111)_2$$

$$\text{bin}_{32}(538623) = (00000000 \ 00001000 \ 00110111 \ 11111111)_2$$

Zweierkomplement

Mit der Erweiterung der Zahlendarstellung um ein Vorzeichen können die ganzen Zahlen (*signed integer*) dargestellt werden.

Da auf Hardware-Ebene mit Bitfolgen fester Länge gearbeitet wird, bietet sich das **Zweierkomplement**, als eine Alternative zum Vorzeichen an.

Für eine Dezimalzahl z ist $\text{bin}_n^2(z)$ die Bitfolge $(x_{n-1} \dots x_0)_2$ der Länge n , die z im Zweierkomplement codiert.

$$z = -\left(x_{n-1} \cdot 2^{n-1}\right) + \sum_{i=0}^{n-2} x_i \cdot 2^i$$

Beispiel

Beispiele

$$\text{bin}_8(-15) = ?$$

Beispiele

$$\text{bin}_8(-15) = ?$$

$$\begin{aligned} -15 &= -128 + 64 + 32 + 16 + 1 \\ &= -2^7 + 2^6 + 2^5 + 2^4 + 2^0 \\ &= (11110001)_2 \end{aligned}$$

$$\text{bin}_8(-15) = (11110001)_2$$

Beispiele

$$\text{bin}_8(-15) = ?$$

$$\begin{aligned} -15 &= -128 + 64 + 32 + 16 + 1 \\ &= -2^7 + 2^6 + 2^5 + 2^4 + 2^0 \\ &= (11110001)_2 \end{aligned}$$

$$\text{bin}_8(-15) = (11110001)_2$$

$$\text{bin}_{32}(-15) = ?$$

Beispiele

$$\text{bin}_8(-15) = ?$$

$$\begin{aligned} -15 &= -128 + 64 + 32 + 16 + 1 \\ &= -2^7 + 2^6 + 2^5 + 2^4 + 2^0 \\ &= (11110001)_2 \end{aligned}$$

$$\text{bin}_8(-15) = (11110001)_2$$

$$\text{bin}_{32}(-15) = (11111111 \ 11111111 \ 11111111 \ 11110001)_2$$

Überlauf

Bei ganzzahligen Typen kann in der Regel nur ein **Überlauf** auftreten, d.h. bei Rechnen entsteht eine Bit, für das kein Platz mehr in dem für die Zahl vorgesehenen Speicherbereich ist.

Überlauf bei ganzzahligen Operationen führt in der Regel nicht zu einem Laufzeitfehler, sondern es wird mit falschen Werten weiter gerechnet.

Eine Ausnahme ist häufig die Divisionen (Rest der Division) mit Divisor Null, die zu einem Programmabbruch führt, durch den Standard festgelegt ist dieses Verhalten aber nicht.

Der Hintergrund für dieses überraschende Verhalten ist die Architektur des Rechenwerks. Bei ganzzahligen Operationen tritt häufiger ein Hardware-Überlauf auf, **ohne** dass es sich dabei um einen Fehler handelt.

Beispiel

Der Einfachheit halber wird 8-Bit-Arithmetik betrachtet.

$$\begin{array}{r} (1111\ 1111)_2 \\ + (0000\ 0001)_2 \\ \hline = (1\ 0000\ 0000)_2 \end{array}$$

Dabei ist das werthöchste, neunte Bit gerade das überlaufende Bit, das nicht mehr gespeichert werden kann. Ob dieser Überlauf ein Fehler ist oder nicht, hängt davon ab, ob die Operanden als vorzeichenbehaftet oder vorzeichenlos betrachtet werden.

- Vorzeichenbehaftet ergibt sich das korrekte Resultat $-1 + 1 = 0$.
- Vorzeichenlos resultiert $255 + 1 = 0$, das ist offenbar falsch.

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Wertebereiche

Der C-Standard schreibt keine festen Wertebereiche, sondern nur **Mindestschranken** für die Wertebereiche vor.

Die **benannten Konstanten** für die konkreten, implementationsspezifischen **Wertebereiche** der ganzzahligen Typen findet man in `limits.h`.

short-Typen

`SHRT_MIN, SHRT_MAX, USHRT_MAX`

int-Typen

`INT_MIN, INT_MAX, UINT_MAX`

long-Typen

`LONG_MIN, LONG_MAX, ULONG_MAX`

long long-Typen

`LLONG_MIN, LLONG_MAX, ULLONG_MAX`

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Beispiel

Beispiel

Berechnen der Fakultätsfunktion.

Beispiel (1/4)

```
1  #include <stdio.h>
2
3  int fac(int n) {
4      if (n < 0)
5          return -1;          // error
6
7      if (n == 0)
8          return 1;
9
10     int f = 1;
11     for (int i = 2; i <= n; i++)
12         f = f*i;
13
14     return f;
15 }
16
17 int main() {
18     for (int i = -1; i < 20; i++)
19         printf("%d!=%d\n", i, fac(i));
20
21     return 0;
22 }
```

Beispiel (2/4)

```
> gcc -o faculty fac.c
> ./faculty
-1!=-1
0!=1
1!=1
2!=2
3!=6
4!=24
5!=120
6!=720
7!=5040
8!=40320
9!=362880
10!=3628800
11!=39916800
12!=479001600
13!=1932053504
14!=1278945280
15!=2004310016
16!=2004189184
17!=-288522240
18!=-898433024
19!=109641728
```

Offenbar kommt es schon bei `fac(14)` zu einem Überlauf, ein Benutzer der Funktion `fac` kann das aber nicht erkennen bzw. abfragen.

Beispiel (3/4)

```
1  #include <stdio.h>
2  #include <limits.h>          // INT_MAX
3
4  int fac(int n) {
5      if (n < 0)
6          return -1;           // error
7
8      if (n == 0)
9          return 1;
10
11     int f = 1;
12     for (int i = 2; i <= n; i++) {
13         if (f > INT_MAX/i)    // prevent overflow
14             return -2;       // error
15         f = f*i;
16     }
17     return f;
18 }
19
20 int main() {
21     for (int i = -1; i < 20; i++)
22         printf("%d!=%d\n", i, fac(i));
23     return 0;
24 }
```


Beispiel (4/4)

```
> gcc -o faculty fac.c
> ./faculty
-1!=-1
0!=1
1!=1
2!=2
3!=6
4!=24
5!=120
6!=720
7!=5040
8!=40320
9!=362880
10!=3628800
11!=39916800
12!=479001600
13!=-2
14!=-2
15!=-2
16!=-2
17!=-2
18!=-2
19!=-2
```

Durch Verhindern des Überlauf und Zurückliefern eines passenden Fehlerwerts, kann ein Benutzer der Funktion `fac` erkennen, dass schon `fac(13)` nicht berechnet werden kann, weil ein Überlauf eintreten würde.

Bemerkung

Die richtige Formulierung der Bedingung ist wichtig.

Die Prüfung von

$$f > \text{INT_MAX} / i$$

vor der Berechnung von $f * i$ verhindert das Eintreten des Überlaufs.

Der Versuch mit

$$f * i > \text{INT_MAX}$$

nach der kritischen Rechnung zu testen, ob eine Überlauf stattgefunden hat, kann nicht gelingen.

Bemerkung. Es gibt keinen `int`-Wert größer `INT_MAX`.

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Anforderungen

Minimalanforderungen für `double`-Zahlen.

Der geforderte minimale Wertebereich ist

$$[-10^{+37}, -10^{-37}] \cup \{0\} \cup [+10^{-37}, +10^{+37}]$$

bei einer minimalen Dichte von 10^{-9} .

Hieraus ergeben sich direkt die drei „problematischen“ Fälle.

- Es kann ein *Überlauf* (**overflow**) eintreten, d.h. ein Wert wird dem Betrage nach größer als `DBL_MAX`.
- Es kann ein *Unterlauf* (**underflow**) eintreten, d.h. ein Wert wird dem Betrage nach kleiner als `DBL_MIN`.
- Ein Wert fällt in eine „Lücke“ zwischen zwei darstellbaren Zahlen, d.h. er muss gerundet werden.

Problematische Fälle

Wie auf solche Fälle reagiert wird, kann von Rechner zu Rechner sehr unterschiedlich sein. Man kann deshalb nur „übliche“ Reaktionen beschreiben; andere Reaktionen sind denkbar.

- Beim Überlauf gibt es drei „übliche“ Reaktionen:
 - ▶ Das Programm wird abgebrochen.
 - ▶ Der Wert wird auf „Infinity“ gesetzt, einen ausgezeichneten Wert.
 - ▶ Der Wert wird undefiniert.

Division durch Null wird mit gewisser Berechtigung in der Regel wie ein Überlauf behandelt.

- Bei Unterlauf ist es üblich, den Wert durch Null zu ersetzen, ohne dass dieses dem Programm angezeigt wird.
- Gerundet wird in der Regel automatisch, ohne dass dieses dem Programm angezeigt wird.

Nur für den Überlauf schreibt C vor, dass ein Programm selbst festlegen kann, wie darauf zu reagieren ist. Die entsprechenden Mittel stellt C mit der Headerdatei `signal.h` zur Verfügung.

Typumwandlungen

Bei den impliziten Typumwandlungen im Zuge von arithmetischen Operationen sind die Regeln gerade so angelegt, dass Überlauf und Unterlauf nicht auftreten können.

Werden Typumwandlungen dagegen durch Wertzuweisungen oder Castoperatoren erzwungen, können Überlauf und Unterlauf wie bei arithmetischen Operationen auftreten.

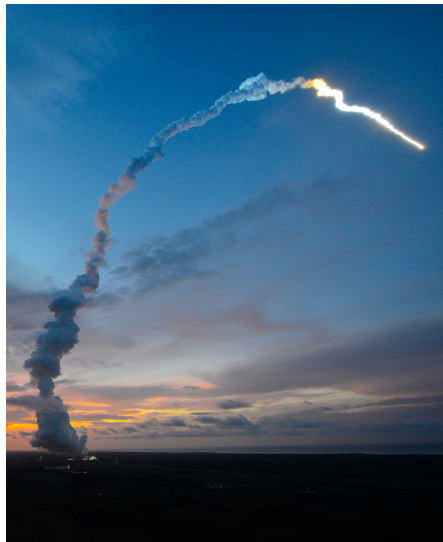
Ariane 5

4. Juni 1996 erster Flug der Ariane 5, nach 40s Selbstzerstörung, Schaden 370 Millionen US-Dollar.

Mehrere Ursachen, u.a. rechnete der Bordcomputer den Wert der horizontalen Geschwindigkeit von 64 Bit Fließkomma in 16 Bit signed Integer um. 36,7s nach dem Start war dieser Wert zu groß und es kam zu einem **Überlauf**, der weder verhindert noch behandelt wurde.

Bordcomputer und Backup-Rechner stützten ab, es folgte die Selbstzerstörung.

Teile der Software waren von der Ariane 4 übernommen worden, diese waren aber für Ariane 5 nicht geeignet und wurden auch nicht umfassend getestet.



Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Rechenregeln

Ausdrücke werden Schritt für Schritt ausgewertet – und in jedem Schritt muss der berechnete Wert innerhalb des Wertebereichs des Typs liegen.

Beispiel Auswertung des Ausdruck $\frac{ab}{cd}$ mit float-Arithmetik.

A	B	C	D	A*B/C/D	A/C*B/D
10^{30}	10^{30}	10^{30}	10^{30}	Überlauf	1
10^{30}	10^{-30}	10^{-30}	10^{30}	1	Überlauf
10^{-30}	10^{-30}	10^{-30}	10^{-30}	Unterlauf	1
10^{-30}	10^{30}	10^{30}	10^{-30}	1	Unterlauf

Obwohl das theoretische Resultat in allen Fällen 1 ist, können Überlauf und Unterlauf auftreten.

Assoziativ- und Kommutativgesetz sind auf einem Rechner für Gleitkommazahlen nicht erfüllt, da die Voraussetzungen für beide Gesetze nicht erfüllt sind. Denn man rechnet nicht in den reellen Zahlen, sondern nur in einer Teilmenge, deren Werte nicht dicht in den reellen Zahlen liegen.

Folgerung

Man muss sich bei Gleitkommaausdrücken genau überlegen, in welcher Reihenfolge diese ausgewertet werden sollen.

Der Standard unterstützt das, im Gegensatz zum „alten“ C. Scheinbar redundante Klammern dürfen nicht mehr „wegoptimiert“ werden. War ein Compiler früher berechtigt, in einem Ausdruck wie

$$a + (b + c)$$

die Klammern schlichtweg zu ignorieren (was unter Umständen zu „Klimmzügen“ im Programm zwang), so **muss** ein Standardcompiler sie jetzt berücksichtigen.

Beispiel

Sind $a = 1$, $b = -1e12$ und $c = 1e12$, dann ergibt sich bei einem Rechenwerk mit 10 signifikanten Stellen

$$a + (b + c) = 1$$

$$(a + b) + c = 0$$

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Rundung

Rundung ist aufgrund der begrenzten Stellenzahl in vielen Fällen unumgänglich, daraus ergebenden sich **Rundungsfehler**.

Angenommen man hat ein dezimales Rechenwerk zur Verfügung, das mit zwei Dezimalstellen Genauigkeit arbeitet. Dann lässt sich in diesem Rechenwerk die Zahl 1.5 exakt speichern.

Bildet man jetzt jedoch das Produkt $1.5 * 1.5$, so resultiert die Zahl 2.25 mit drei signifikanten Stellen, die nicht mehr exakt gespeichert werden kann, sondern zunächst durch Rundung auf zwei signifikante Stellen verkürzt werden muss.

Im angenommen Rechenwerk ist das Resultat, je nach Art der Rundung, also entweder 2.2 oder 2.3.

Genau diese Probleme treten bei jedem Rechner auf. Allerdings lässt sich das, was bei einem Rechner passiert, für einen Menschen nicht mehr so auf den ersten Blick überschauen.

Vielfach wird übersehen, dass Rundungsfehler nicht erst beim eigentlichen Rechnen, sondern bereits bei der Umwandlung der dezimalen Konstantendarstellung in die interne Darstellung auftreten.

Beispiel

Der „schöne“ Dezimalbruch 0.2_d ergibt bei Umwandlung in Binärdarstellung den periodischen Bruch $0.001100110011 \dots_b$, dieser lässt sich im Rechner nur gerundet darstellen.

Rundung

Bei arithmetischen Operationen mit gerundeten Zahlen können die Rundungsfehler unter Umständen erheblich anwachsen.

Beispiel

Der Wert 0.2 wird 10000 Mal addiert und das Ergebnis ausgegeben.

```
float step = 0.2f, sum = 0.0f;
int i;

for (i = 0; i < 10000; i++)
    sum += step;

printf("%f\n", sum);
printf("%f\n", 10000 * step);
```

Ein Testlauf ergibt für die erste Ausgabe den Wert 1999.805786 und für die zweite Ausgabe 2000.000000.

Rundung

Ein Programmierer muss sich sehr genau überlegen, wie er sein Programm formuliert, um den Einfluss der Rundungsfehler möglichst gering zu halten.

Beispiel

Eine Funktion $f(x)$ sei in einem Intervall $[a, b]$ zu tabellieren, wobei die Anzahl N der Teilintervalle vorgegeben ist.

Als “naive” Lösung bietet sich an, zunächst einmal die Schrittweite zu berechnen und ihren Wert dann bei jedem Schleifendurchlauf zur Bestimmung des nächsten Tabellierungspunktes zu verwenden.

Das Programm hat dann das folgende Schema.

```
h = (b - a) / N;  
x = a;  
while (x <= b) {  
    printf("%f %f\n", x, f (x));  
    x += h;  
}
```

Mit den Werten $a = 0$, $b = 4$ und $N = 40$ erhält hier die Variable x vor dem letzten Schleifendurchlauf den Wert 3.9999980, erreicht also nicht exakt den rechten Rand des Tabellierungsintervalls. Entsprechend ungenau werden natürlich auch die tabellierten Funktionswerte.

Rundung

Günstiger ist es, nicht mit der Schrittweite zu arbeiten, sondern die Tabellierungspunkte jeweils direkt zu berechnen, etwa nach dem folgenden Schema.

```
i = 0;
while (i <= N) {
    x = ((N - i) * a + i * b) / N;
    printf("%f %f\n", x, f (x));
    i++;
}
```

Hier wird, wieder mit den Werten $a = 0$, $b = 4$ und $N = 40$, beim letzten Schleifendurchlauf der rechte Rand des Tabellierungsintervalls exakt erreicht.

Rundung

Auch bei Typumwandlungen können Rundungsfehler auftreten – und das nicht nur bei erzwungenen Umwandlungen.

Für den Typ `float` sind nur 6 signifikante Stellen vorgeschrieben, während der Typ `long` über mindestens 10 Stellen verfügen muss.

Bei einer Umwandlung von `long` in `float`, wie sie ggf. automatisch bei der Auswertung von Ausdrücken erfolgt, können also durchaus die letzten Stellen des `long`-Wertes verloren gehen.

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Auslöschung (1/3)

Auslöschung ist das zwischenzeitliche Vorkommen führender Nullen.

Man kann die Auslöschung in bestimmten Situationen beherrschen oder zumindest reduzieren. In anderen Situationen ist das nicht so einfach oder sogar unmöglich.

Beispiel

5-stellige dezimale Gleitkomma-Arithmetik.

$$x_1 = 0.42178 \cdot 10^2, x_2 = 0.23722 \cdot 10^3, y_1 = 0.99986 \cdot 10^2, y_2 = 0.99987 \cdot 10^2$$

Dann gilt

$$x_1 x_2 = 0.1000546516 \cdot 10^5$$

$$y_1 y_2 = 0.9997300182 \cdot 10^4$$

und

$$\begin{aligned} x_1 x_2 - y_1 y_2 &= 0.10005465160 \cdot 10^5 \\ &\quad - 0.09997300182 \cdot 10^5 \\ &\quad \hline &= 0.00008164978 \cdot 10^5 \end{aligned}$$

Ergebnis in 5 Dezimalstellen $0.81649 \cdot 10^1$ oder $0.81650 \cdot 10^1$, abhängig vom Rundungsverfahren.

Auslöschung (2/3)

Erlaubt die verfügbare Arithmetik nicht die exakte Darstellung der Zwischenresultate, sondern z.B. nur fünfstellige Resultate, so werden bereits die Produkte x_1x_2 und y_1y_2 gerundet geliefert.

$$x_1x_2 \approx x = 0.10005 \cdot 10^5 \quad \text{und} \quad y_1y_2 \approx y = 0.99973 \cdot 10^4$$

Erst danach kann die Subtraktion ausgeführt werden und liefert

$$\begin{array}{rcl} x - y & = & 0.100050 \cdot 10^5 \\ & - & 0.099973 \cdot 10^5 \\ \hline & = & 0.000077 \cdot 10^5 \end{array}$$

Ergebnis in 5 Dezimalstellen $0.77000 \cdot 10^1$.

Auslöschung (3/3)

Der Vergleich der Differenzen $x_1x_2 - y_1y_2$ und $x - y$ zeigt, dass das gerundete Resultat zwar dieselbe Größenordnung besitzt wie das exakte Resultat, dass aber nicht einmal die werthöchsten Ziffern übereinstimmen.

Auslöschung tritt immer dann ein, wenn zwei beinahe gleichgroße Zahlen subtrahiert werden. Selbst wenn die einzelne Subtraktion fehlerfrei ausgeführt wird, ist die Auslöschung numerisch gefährlich, wenn die zu subtrahierenden Daten selber bereits gerundet wurden.

Taylorreihe (1/2)

Im gerade betrachteten Beispiel hatte das tatsächliche Resultat noch die Größenordnung des theoretischen Resultats. Es bereitet jedoch keine Probleme, Beispiele zu finden, in denen auch die Größenordnungen voneinander abweichen.

Dazu betrachtet man die Taylorreihe der Exponentialfunktion.

$$e^z = \sum_{k=0}^{\infty} \frac{z^k}{k!}$$

Diese Reihe ist absolut konvergent für jedes z aus der komplexen Zahlenebene.

Für reelles, negatives z haben die Summanden alternierende Vorzeichen. In diesem Fall ist der Fehler, den man begeht, wenn man die n -te Partialsumme als Wert für e^z nimmt, kleiner als der $(n+1)$ -te Summand, der als erster vernachlässigt wird.

Taylorreihe (2/2)

Wertet man diese Reihe unter Verwendung einer Arithmetik mit 6-stelliger dezimaler Mantisse für $z = -14$ aus, so erhält man nach 50 Additionen den Wert -0.159691 – das korrekte Resultat wäre 0.00000083152900 .

Der Abbruch der Summation erfolgt nach 50 Schritten, weil alle weiteren Summanden so klein sind, dass diese bei der verwendeten Arithmetik keinen Beitrag mehr zu der Summe liefern.

Hier lässt sich mit Mitteln der Programmiertechnik nichts mehr ausrichten. Abhilfe schaffen nur noch besondere Algorithmen, die die Probleme der Gleitkommaarithmetik berücksichtigen, oder der Einsatz der Intervallrechnung.

Rechnerarithmetik

Einleitung

Ganze Zahlen

Wiederholung

Überlauf verhindern

Gleitkomma–Ausnahmebehandlung

Assoziativ– und Kommutativgesetz

Rundungsfehler

Auslöschung

Implementation Taylorreihe

Taylorreihe, 1. Implementation (1/2)

```
1  #include <float.h>
2
3  double exp_taylor(double z) {
4      //  $e^z = \sum_k z^k/k!$ 
5      double k = 0.0;
6
7      // for k = 0
8      double pow = 1.0;      //  $z^k$ 
9      double fac = 1.0;      //  $k!$ 
10     double add = 1.0;       //  $z^k/k!$ 
11     double series = 1.0;    //  $\sum_k (z^k/k!)$ 
12
13     for(int i = 0; i < 500; i++) {
14         k += 1.0;
15
16         fac *= k;
17         pow *= z;
18         add = pow/fac;
19         series += add;
20     }
21
22     return series;
23 }
```

Taylorreihe, 1. Implementation (2/2)

```
> gcc -o taylor taylor01.c
> ./taylor
exp(0.000000)=1.000000
exp(1.000000)=2.718282
exp(2.000000)=7.389056
exp(3.000000)=20.085537
exp(4.000000)=54.598150
exp(5.000000)=-nan
exp(6.000000)=-nan
exp(7.000000)=-nan
exp(8.000000)=-nan
exp(9.000000)=-nan
```

Die Berechnung des Bruchs $z^k/k!$ durch Bestimmung von Zähler und Nenner ist offenbar keine gute Idee.

Alternativ kann der Bruch der $(k+1)$ -ten Iteration direkt aus dem Bruch der k -ten Iteration berechnet werden.

$$\frac{z^{k+1}}{(k+1)!} = \frac{z^k}{k!} \cdot \frac{z}{(k+1)}$$

Taylorreihe, 2. Implementation (1/3)

```
1  #include <float.h>
2
3  double exp_taylor(double z) {
4      //  $e^z = \sum_k z^k/k!$ 
5      double k = 0.0;
6
7      // for k = 0
8      double add = 1.0;      //  $z^k/k!$ 
9      double series = 1.0;   //  $\sum_k (z^k/k!)$ 
10
11     for(int i = 0; i < 500; i++) {
12         k += 1.0;
13
14         add *= z;
15         add /= k;
16         series += add;
17     }
18
19     return series;
20 }
```

Bemerkung

Zur Kontrolle kann in der `main`-Funktion, die Funktion `exp` aus der Mathematik-Bibliothek verwendet werden. Der Prototyp ist in `math.h` hinterlegt, die Bibliothek wird mit `-lm` eingebunden.

Taylorreihe, 2. Implementation (2/3)

```
> gcc -o taylor taylor02.c -lm
> ./taylor
```

x	taylor(x)	exp(x)
0.00	1.000000e+00	1.000000e+00
1.00	2.718282e+00	2.718282e+00
2.00	7.389056e+00	7.389056e+00
3.00	2.008554e+01	2.008554e+01
4.00	5.459815e+01	5.459815e+01
5.00	1.484132e+02	1.484132e+02
6.00	4.034288e+02	4.034288e+02
...		
28.00	1.446257e+12	1.446257e+12
29.00	3.931334e+12	3.931334e+12
30.00	1.068647e+13	1.068647e+13
...		
-22.00	-1.973166e-09	2.789468e-10
-21.00	-3.164860e-09	7.582560e-10
-20.00	5.621884e-09	2.061154e-09
-19.00	8.873234e-09	5.602796e-09
-18.00	1.538542e-08	1.522998e-08
-17.00	4.123088e-08	4.139938e-08
...		
-5.00	6.737947e-03	6.737947e-03
-4.00	1.831564e-02	1.831564e-02
-3.00	4.978707e-02	4.978707e-02

Taylorreihe, 2. Implementation (3/3)

Die Abweichung bei negativen Exponenten resultiert aus der *Auslöschung*, die bei der Addition von Partialsumme und Bruch mit unterschiedlichen Vorzeichen auftritt.

Diesen Algorithmus kann man so modifizieren, dass dieses Problem der Gleitkommaarithmetik berücksichtigt wird, denn es gilt $e^{-z} = 1/e^z$.

Taylorreihe, 3. Implementation (1/3)

```
1  #include <float.h>
2
3  double exp_taylor(double z) {
4      //  $e^{-z} = 1/e^z$ 
5      int invert = z < 0.0;
6      if (invert)
7          z = -z;
8
9      //  $e^z = \sum_k z^k/k!$ 
10     double k = 0.0;
11
12     // for k = 0
13     double add = 1.0;      //  $z^k/k!$ 
14     double series = 1.0;   //  $\sum_k (z^k/k!)$ 
15
16     for(int i = 0; i < 500; i++) {
17         k += 1.0;
18         add *= z;
19         add /= k;
20         series += add;
21     }
22
23     if (invert)
24         series = 1/series;
25
26     return series;
27 }
```

Taylorreihe, 3. Implementation (2/3)

```
> gcc -o taylor taylor03.c -lm
> ./taylor
```

x	taylor(x)	exp(x)
0.00	1.000000e+00	1.000000e+00
1.00	2.718282e+00	2.718282e+00
2.00	7.389056e+00	7.389056e+00
3.00	2.008554e+01	2.008554e+01
4.00	5.459815e+01	5.459815e+01
5.00	1.484132e+02	1.484132e+02
6.00	4.034288e+02	4.034288e+02
...		
28.00	1.446257e+12	1.446257e+12
29.00	3.931334e+12	3.931334e+12
30.00	1.068647e+13	1.068647e+13
...		
-22.00	2.789468e-10	2.789468e-10
-21.00	7.582560e-10	7.582560e-10
-20.00	2.061154e-09	2.061154e-09
-19.00	5.602796e-09	5.602796e-09
-18.00	1.522998e-08	1.522998e-08
-17.00	4.139938e-08	4.139938e-08
...		
-5.00	6.737947e-03	6.737947e-03
-4.00	1.831564e-02	1.831564e-02
-3.00	4.978707e-02	4.978707e-02

Taylorreihe, 3. Implementation (3/3)

Diese Lösung hat noch den Nachteil, dass eine feste Anzahl an Iterationen durchlaufen wird, obwohl die dabei berechneten Brüche, das Ergebnis nicht mehr (signifikant) verändern.

Ein gutes Kriterium, um für eine (konvergierende) Reihe die Berechnung zu beenden, kann über dem Abstand von zwei aufeinanderfolgenden Partialsummen formuliert werden.

Unterscheiden sich die n -te Partialsumme $\sum_{k=0}^n z^k/k!$ und die $(n+1)$ -Partialsumme $\sum_{k=0}^{n+1} z^k/k!$ nicht mehr (signifikant) voneinander, wird der Wert der $(n+1)$ -Partialsumme als bestmögliche Näherung für den Grenzwert der Reihe $\sum_{k=0}^{\infty} z^k/k!$ angenommen.

Taylorreihe, 4. Implementation (1/3)

Einleitung und Nachwort bleiben unverändert.

```
1  #include <float.h>
2
3  double exp_taylor(double z) {
4      //  $e^{-z} = 1/e^z$ 
5      int invert = z < 0.0;
6      if (invert)
7          z = -z;
8
9      //  $e^z = \sum_k z^k/k!$ 
10     double k = 0.0;
11
12     // for k = 0
13     double add = 1.0;      //  $z^k/k!$ 
14     double series = 1.0;   //  $\sum_k (z^k/k!)$ 
```

```
33
34     if (invert)
35         series = 1/series;
36
37     return series;
38 }
```

Taylorreihe, 4. Implementation (2/3)

Die Schleife mit angepasster Abbruchbedingung.

```
16 // helper
17 double last = series;    // sum_(k-1)
18
19 int stop = 0;
20 while(!stop) {
21     k += 1.0;
22
23     add *= z;
24     add /= k;
25
26     last = series;
27     series += add;
28
29     // termination condition
30     // distance between the current and the last finite sum
31     stop = add < DBL_EPSILON;
32 }
```

Taylorreihe, 4. Implementation (3/3)

Die richtige Formulierung der Abbruchbedingung ist wichtig, weil *Assoziativ- und Kommutativgesetz* nicht gelten.

```
(series-last < DBL_EPSILON) != (series < last+DBL_EPSILON)
```

Aus $z > 0$ folgt $\sum_{k=0}^n z^k/k! > 1$ für alle $n > 0$, deshalb ist der Vergleich des Abstands der Partialsummen mit `DBL_EPSILON` sinnvoll, den der Abstand zweier `double`-Zahlen größer Eins ist mindestens die Dichte `DBL_EPSILON`.

Um auf Nummer sicher zu gehen, müsste noch getestet werden, dass beim Update von `add` kein Überlauf/Unterlauf eintritt.